PRACTICAL-1

AIM: Write a program to conduct uninformed and informed serach.

Uninformed search:

The uninformed search is also known as a blind search. In this algorithm there is no additional information is available to complete the search process. There is only initial and goal is given to complete the process. The uninformed category is further divided into two types depth (i) depth first search (ii) breadth first search.

1. DFS (depth-first search):

Depth first search (DFS) algorithm is implementing by the concept of Stack. The logic of Stack is LIFO (last in first out) and there are two operations push and pop.

Push: push operation is used to insert the elements into the stack.

Pop: pop operation is used for deleting the elements from the stack.

CODE:

```
#include <iostream>

#include <vector>

#include <stack> using

namespace std;

// Function to perform DFS on a graph

void DFS(int start, const vector<vector<int>>& graph, vector<bool>& visited) {

    // initialize an empty stack s

   stack<int> s;

    // push start  in stack s.push(start);

   // Mark the starting node as visited i.e make visited[start] true visited[start] =

   true;

   // till stack is not empty while

   (!s.empty()) {

       // store top of stack into node int

       node = s.top();

       // pop the node
```

```
        s.pop();
        // print the popped node
        cout << node << " ";
        // Explore all adjacent nodes
        // for neighbour in graph[node]
        for (int neighbor : graph[node]) {
            // if neighbour are not visited i.e !=visited[neighbour]
            if (!visited[neighbor]) {
                // push the neighbour
                s.push(neighbor);
                // mark the neighbour as visited
                visited[neighbor] = true;
            }
        }
    }
}
int main() {
    // Number of nodes in the graph
    int nodes = 6;
    // vector of vector
    vector<vector<int>> graph(nodes);
    // Creating a sample graph (undirected)
    graph[0] = {1, 2};
    graph[1] = {0, 3, 4};
    graph[2] = {0, 4};
    graph[3] = {1, 5};
    graph[4] = {1, 2, 5};
    graph[5] = {3, 4};
    // Vector to track visited nodes
    vector<bool> visited(nodes, false);
```

```
    // Starting DFS from node 0
    cout << "DFS starting from node 0: ";
    DFS(0, graph, visited);
    return 0;
}
```

OUTPUT:

```
#include <iostream>
#include <vector>
#include <stack>
using namespace std;
// Function to perform DFS on a graph
void DFS(int start, const vector<vector<int>>& graph, vector<bool>&
    visited) {
     // initialize an empty stack s
    stack<int> s;
     // push start  in stack
    s.push(start);
    // Mark the starting node as visited i.e make visited[start] true

    visited[start] = true;
    // till stack is not empty
    while (!s.empty()) {
        // store top of stack into node
        int node = s.top();
        // pop the node
        s.pop();
        // print the popped node
        cout << node << " ";
        // Explore all adjacent nodes
```

```
DFS starting from node 0: 0 2 4 5 3 1

=== Code Execution Successful ===
```

2. BFS (breadth-first search):

Breadth-first search (BFS) algorithm is implementing by the concept of Queue. The logic of Queue is FIFO (first in first out) and there are two operations enqueue and dequeue.

Enqueue: It is used to insert the elements into the queue.

Dequeue: It is used for deleting the elements from the queue.

CODE:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
// Function to perform BFS on a graph
void BFS(int start, const vector<vector<int>>& graph, vector<bool>& visited) {
    // initialize an empty queue q
    queue<int> q;
    // enque start  in queue use inbuilt function push
```

```
    q.push(start);
    // Mark the starting node as visited i.e make visited[start] true
    visited[start] = true;
    // till queue is not empty
    while (!q.empty()) {
        // store front of queue into node
        int node = q.front();
        // Deque the node use inbuilt function pop for that
        q.pop();
        // print the deque node
        cout << node << " ";
        // Explore all adjacent nodes
        // for neighbour in graph[node]
        for (int neighbor : graph[node]) {
            // if neighbour are not visited i.e !=visited[neighbour]
            if (!visited[neighbor]) {
                // push the neighbour
                q.push(neighbor);
                // mark the neighbour as visited
                visited[neighbor] = true;
            }
        }
    }
}
int main() {
    // Number of nodes in the graph
    int nodes = 6;
    // vector of vector
    vector<vector<int>> graph(nodes);
    // Creating a sample graph (undirected)
```

```
    graph[0] = {1, 2};
    graph[1] = {0, 3, 4};
    graph[2] = {0, 4};
    graph[3] = {1, 5};
    graph[4] = {1, 2, 5};
    graph[5] = {3, 4};
    // Vector to track visited nodes
    vector<bool> visited(nodes, false);
    // Starting BFS from node 0
    cout << "BFS starting from node 0: ";
    BFS(0, graph, visited);
    return 0;
}
```

OUTPUT:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
// Function to perform BFS on a graph
void BFS(int start, const vector<vector<int>>& graph, vector<bool>&
    visited) {
    // initialize an empty queue q
    queue<int> q;
    // enque start  in queue use inbuilt function push
    q.push(start);
    // Mark the starting node as visited i.e make visited[start] true
    visited[start] = true;
    // till queue is not empty
    while (!q.empty()) {
        // store front of queue into node
        int node = q.front();
        // Deque the node use inbuilt function pop for that
        q.pop();
        // print the deque node
        cout << node << " ";
        // Explore all adjacent nodes
        // for neighbour in graph[node]
```

```
BFS starting from node 0: 0 1 2 3 4 5

=== Code Execution Successful ===
```